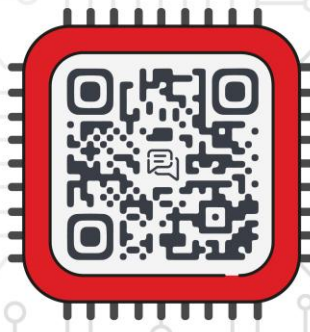




3



16



2400 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

Heap



د. مدحت الصّوص

RB Informatics; 26/04/2026

محتوى مجاني غير مخصص للبيع التجاري

خوارزميات ٢

في المحاضرات السابقة تحدّثنا عن الأشجار (Trees) و عن الشجرة الثنائية و شجرة البحث الثنائية (BST) و تحدّثنا أيضاً عن طرق تمثيلها برمجياً ومنها التخزين في مصفوفة أحادية و كانت المشكلة فيها هي هدر الذاكرة بسبب وجود فراغات في المصفوفة.

و اليوم سنتحدّث عن الـ Heap (الكومة)، و هي بنية (كومة) بيانات تعتمد على فكرة الشجرة الثنائية وتخزن في مصفوفة أحادية.

أهم العناوين الواردة في المحاضرة	
رقم الصفحة	عنوان الفقرة
2	الكومة Heap
2	Binary Heap و أنواعه: Max-Heap / Min-Heap
10	تطبيقات و استخدامات الـ Heap

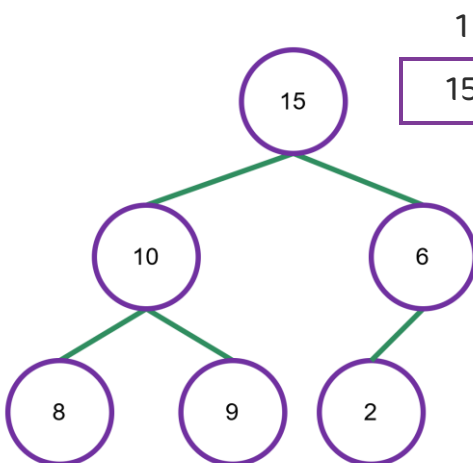
الـ Heap

هي بنية (كومة) بيانات تعتمد على فكرة الشجرة الثنائية و تخزن في مصفوفة أحادية. ما يميز الـ Heap أن الشجرة الثنائية تكون مكتملة (أي Complete Tree)، و هي الشجرة التي تكون جميع مستوياتها ممتلئة ما عدا المستوى الأخير فقد يحوي فراغات من جهة **اليمين حصراً**.

يتم تمثيل الـ Binary Heap في مصفوفة أحادية لا فراغات فيها، ونراعي فيها الآتي:

- أول عنصر في المصفوفة هو الجذر (Root) عند $index = 1$
- الابن اليساري لكل عقدة يقع في الفهرس: $index = 2i$
- الابن اليميني لكل عقدة يقع في الفهرس: $index = 2i + 1$

مثال: ليكن لدينا الـ Heap الآتية والممثلة في مصفوفة:



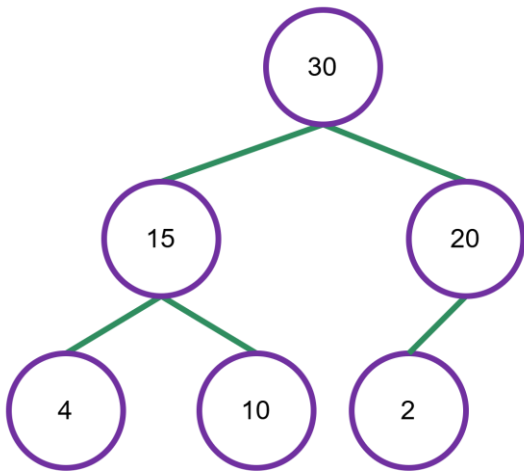
1	2	3	4	5	6
15	10	6	8	9	2

- العنصر الأول (15): هو الجذر في $i = 1$
- أبناء الـ (15):
 - الابن اليساري: (10)
 - الابن اليميني: $index = 2i + 1 = 2(1) + 1 = 3$
- الابن اليميني: 6

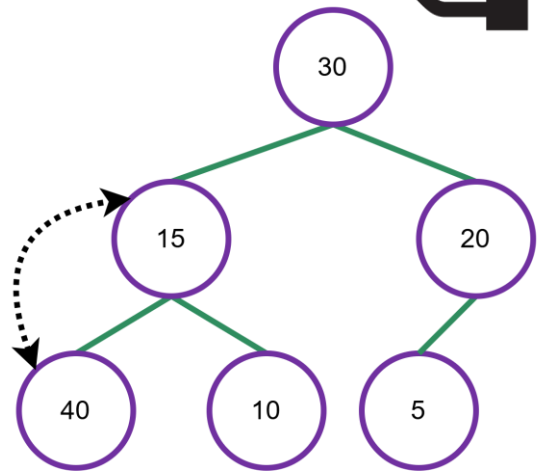
أنواع الـ "Binary Heap"

1. Max-Heap:

هي Binary Heap عادية لكنها مقيدة بالشرط الآتي: يجب أن تكون قيمة كل عقدة أب أكبر من قيم جميع أبنائها و أحفادها (نلاحظ أن الجذر هو أكبر عنصر في الشجرة)



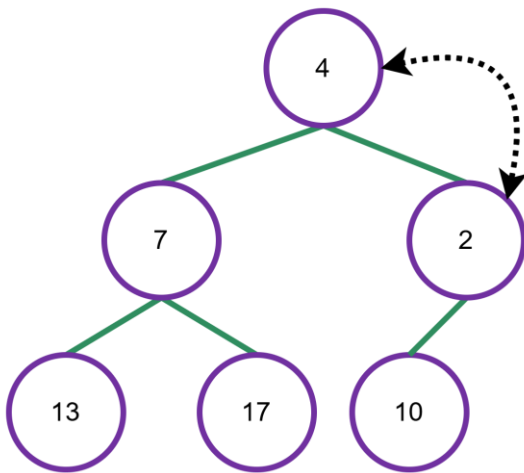
هذه Max-Heap
لأن كل عقدة أكبر من جميع أبنائها



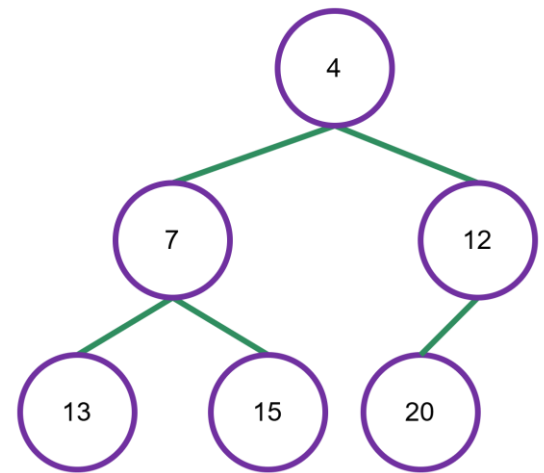
هذه ليست Max-Heap
لأن $40 > 15$ فالشرط غير محقق

2. Min-Heap:

هي مثل Max-Heap لكن سنعكس الشرط حيث ستكون قيمة كل عقدة أب أصغر من جميع أبنائها و أحفادها (نلاحظ أن الجذر هو أصغر عنصر في الشجرة)



هذه ليست Min-Heap
لأن $2 < 4$ فالشرط غير محقق



هذه Min-Heap
لأن كل عقدة أصغر من جميع أبنائها

ملاحظات هامة:

1. المصفوفة الأحادية التي تمثل Max-Heap ليس من الضروري أن تكون مرتبة و لكن إذا كانت مرتبة فهي حتماً Max-Heap (لأن المصفوفة المرتبة في هذه الحالة خضعت لشرط إضافي و هو ترتيب العقد الأخوة -الموجودة في المستوى الواحد- لذلك لا نهتم بالمصفوفة المرتبة بالكامل فكل ما يهمنا هو أن تكون المصفوفة مرتبة جزئياً؛ أي أن الترتيب في علاقة الأب مع أبنائه و أحفاده فقط).

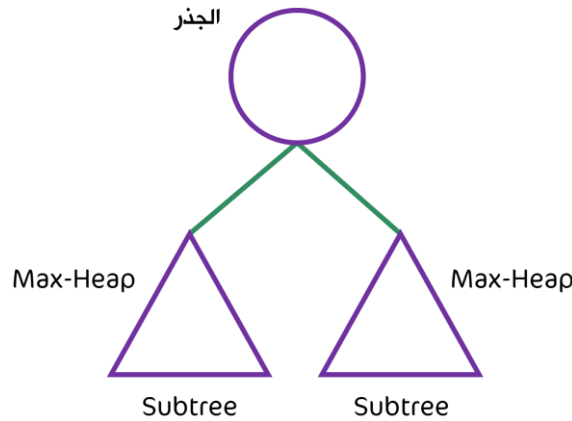
2. نستفيد من Max-Heap في إيجاد أكبر عنصر في المصفوفة بتعقيد $O(1)$.
 3. نستفيد من Min-Heap في إيجاد أصغر عنصر في المصفوفة بتعقيد $O(1)$.
- التطبيقات التي سنتناولها في محاضرتنا عن Max-Heap يمكن أن تطبق أيضاً على Min-Heap مع الانتباه على تحقيق الشرط (العقدة الأب أصغر من أبنائها).

سؤال:

لو كان لدينا Subtree تشكّل Max-Heap باستثناء الجذر؛ أي نحن متأكدين أنّ كلّ subtree هي Max-Heap محققة للشرط لكنّ لسنا متأكدين أنّ الجذر يحقق هذا الشرط أم لا

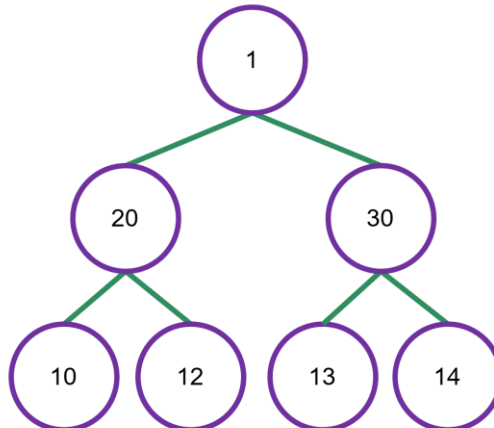
هنا سنميز حالتين:

1. الجذر أكبر من أبنائه: فهو يحقق شرط Max-Heap.
2. الجذر أصغر من أبنائه: فهو لا يحقق الشرط و يجب علينا إصلاح الخلل باستخدام خوارزمية (Max-Heap) لنحصل على شجرة تحقق Max-Heap.

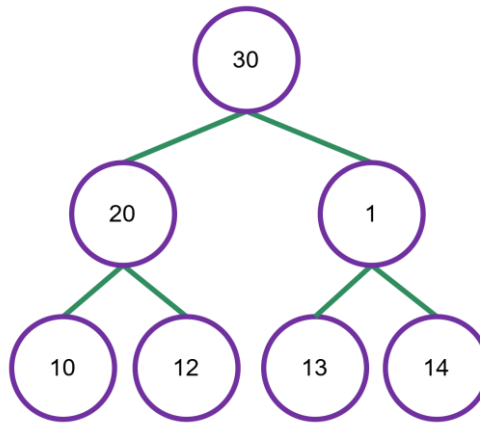


خوارزمية Max-Heapify (إصلاح الخلل):

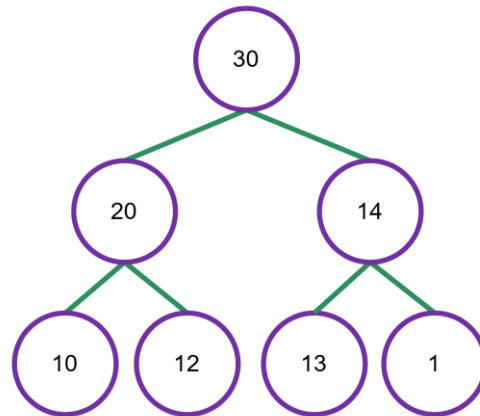
ليكن لدينا الشجرة الثنائية ونهدف لتحويلها إلى Max-Heap باعتبار أنّ كلّاً من subtree اليميني و subtree اليساري للعقدة الجذر هي Max-Heap.



- لنقارن الجذر مع ابنه (20) و (30) و نستبدله بالابن الأكبر (30)



- بعد تبديل العقدة نلاحظ وجود خلل في subtree اليميني فهو لا يحقق شرط Max-Heap لذلك؛ سنقوم بعملية الاستبدال ذاتها بين الأب (1) و ابنه الأكبر (14).
فنحصل على الشجرة و قد حققت شرط Max-Heap على كل subtree فيها.



ملاحظة:

متى تتوقف عملية الاستبدال بين العقد ؟

عند الوصول إلى ورقة (نهاية الشجرة) أو حتّى الوصول إلى subtree تحقق شرط Max-Heap.

```

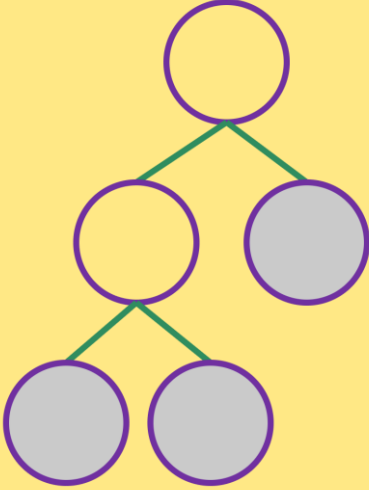
1 void Max_Heapify (int [] A, int i){
2     int left = 2*i;
3     int right = 2*i + 1;
4     int largest = i;
5     if (left <= A.length && A[left] > A[largest])
6         largest = left;
7     if (right <= A.length && A[right] > A[largest])
8         largest = right;
9     if (largest != i)
10        swap(A[i], A[largest]);
11    Max_Heapify (A, largest);
12 }
  
```

ما هو تعقيد الخوارزمية ؟

التعقيد من رتبة الارتفاع $O(H)$ ، و بما أن الشجرة Binary tree فكما تعلّمنا سابقاً هو من رتبة $O(\log_2 n)$.

ملاحظة هامة جداً !!

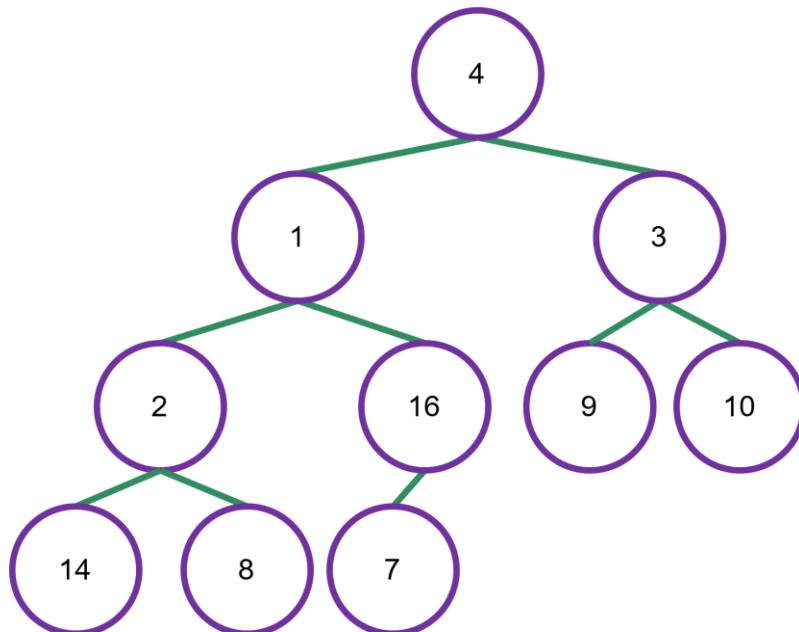
عدد العقد الداخليّ يساوي عدد الأوراق أو أقلّ منها بواحد؛
و بالتالي فإنّ الأوراق تشكّل:
"نصف الشجرة" أو "نصف الشجرة + 1".



تحويل شجرة عشوائية إلى Max-Heap بناء Max-Heap:

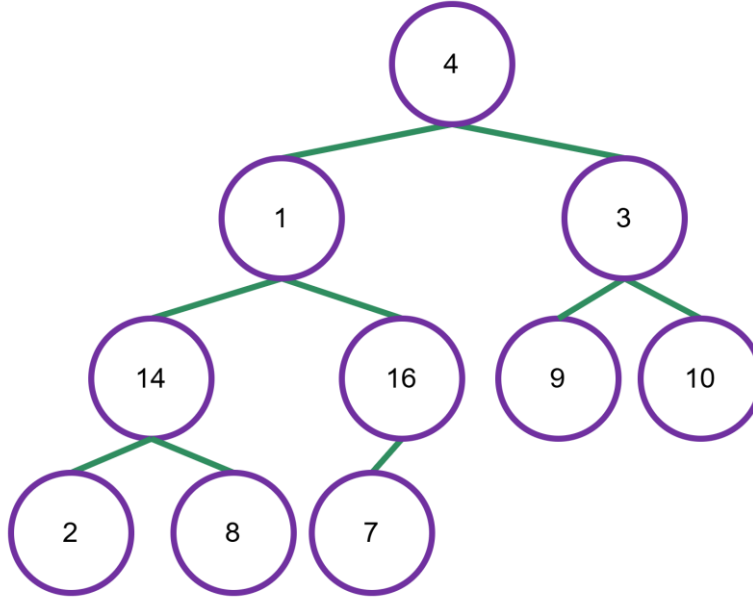
- نبدأ من أسفل الشجرة (أي نبدأ عملية الترتيب من آباء الأوراق).
- نطبّق خوارزمية Max-Heapify على كلّ مستوى في الشجرة.
- نصعد من الأسفل و حتى الوصول إلى الجذر لنحصل على شجرة هي Max-Heap.

مثال: لدينا الشجرة:



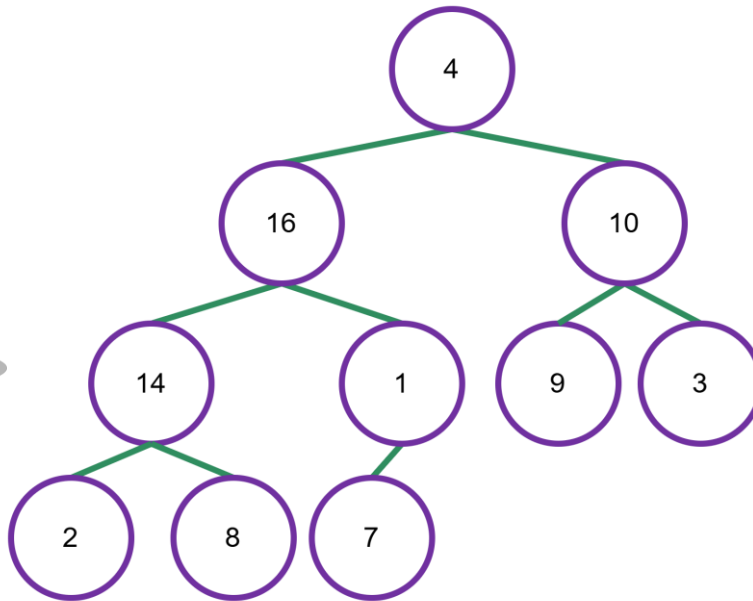
نبدأ من آخر عقدة داخلية وهما (16) و (2).

- العقدة (16) أكبر من أبنائها ← لا نبذل.
 - العقدة (2) أصغر من أبنائها ← نبذل العقدة (2) مع الابن الأكبر و هو الابن اليساري (14).
- فنحصل على:

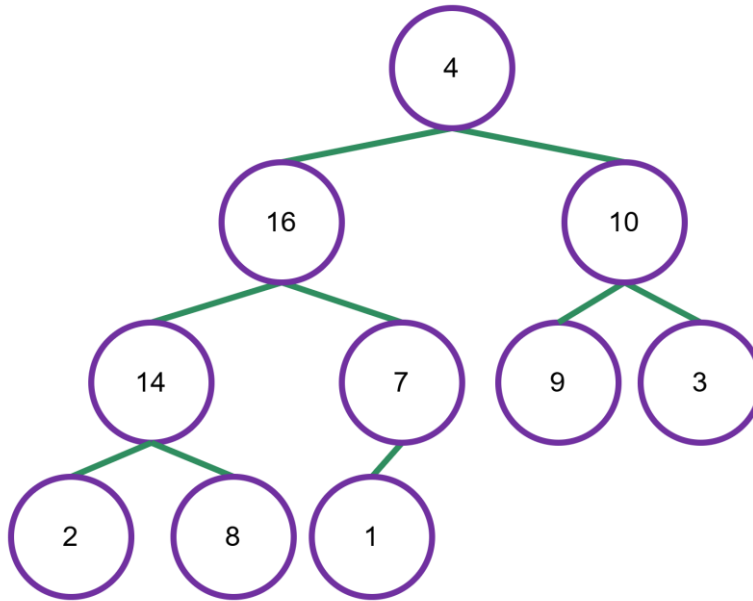


ننتقل للمستوى الأعلى أي العقدتين (3) و (1).

- العقدة (3) أصغر من أبنائها ← نبذل العقدة (3) مع ابنها الأكبر و هو الابن اليميني (10)
 - العقدة (1) أصغر من أبنائها ← نبذل العقدة (1) مع ابنها الأكبر و هو الابن اليميني (16)
- فنحصل على:

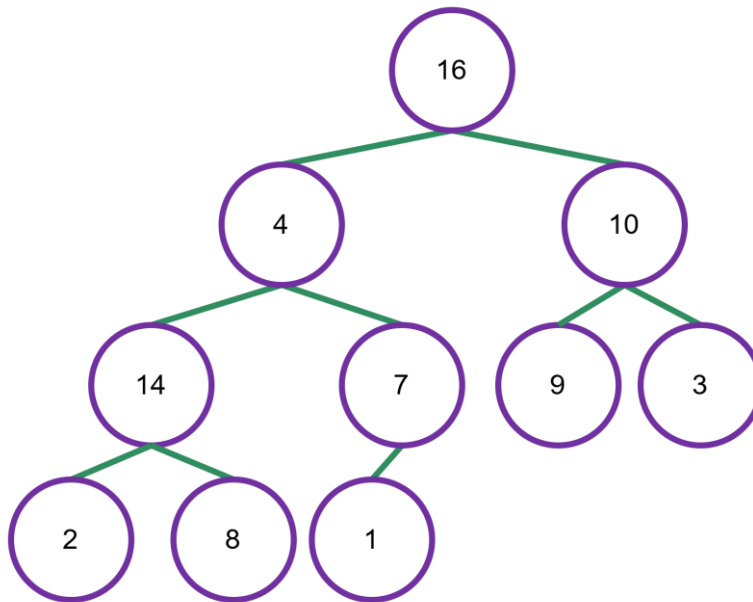


نلاحظ أنه بعد تبديل العقدة (1) حدث خلل في الـ Subtree و اختل شرط Max-Heap و لإصلاح الخلل سنبدّل بين العقدة (1) و العقدة (7) التي هي الابن الوحيد الأكبر منها.



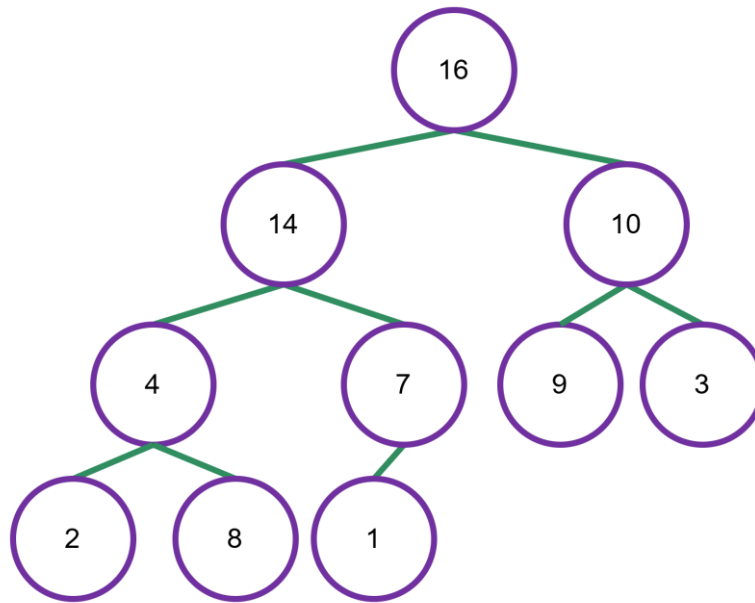
ننتقل الآن إلى المستوى الأعلى حيث العقدة (4) التي تمثل الجذر و هي أصغر من ابنيها (10) و (16): إذاً سنبدّلها مع الأكبر و هو العقدة (16).

فنحصل على:



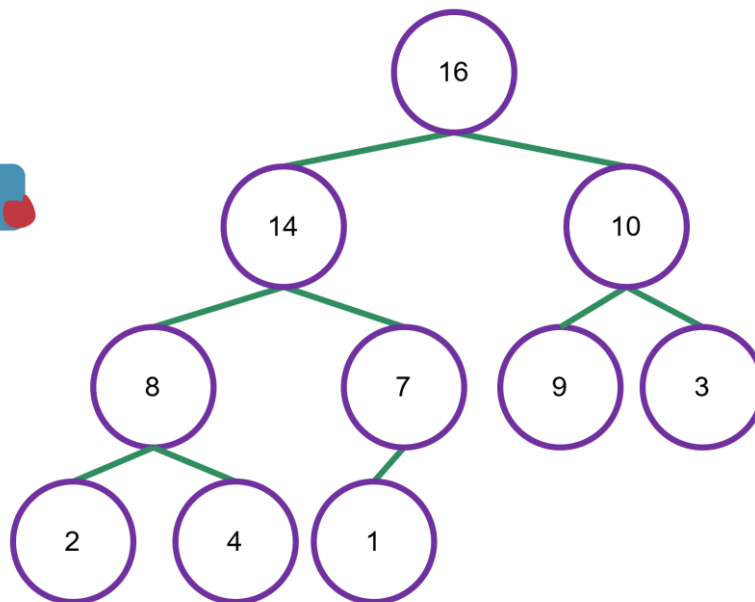
نلاحظ أنه بعد التبديل حدثَ خلل آخر في العقدة (4) لذلك سنبدل العقدة (4) مع أكبر ابنيها و هو العقدة (14).

فنحصل على:



نلاحظ أنّ الخلل لم يتمّ إصلاحه بالكامل؛ لأنّ العقدة (4) ما زالت أصغر من أحد أبنائها و هو العقدة اليمينية (8) لذلك؛ سنبدل بين العقدين (4) و (8)

فنحصل على:



و هكذا نكون قد حصلنا على شجرة Max-Heap بالكامل.

التعقيد هو:

(عدد العقد التي طبقنا عليها Max-Heapify) × (تعقيد Max-Heapify)

و نعلم أن عدد العقد التي طبقنا عليها Max-Heapify هي $\frac{n}{2}$ لأننا لا نطبق Max-Heapify على الأوراق.

و كما ذكرنا في الملاحظة السابقة فإن عدد الأوراق في الـ Complete Tree هو نصف عدد العقد في الشجرة.

و بالتالي فإن التعقيد:

$$O\left(\frac{n}{2} \log_2 n\right) = O(n \log_2 n)$$

⚡ كود الخوارزمية (Build Max-Heap):

```
1 void build_Max-Heap (int[] A) {
2     for (int i = A.length/2; i > 0; i--)
3         Max_Heapify (A, i);
4 }
```

شرح الكود:

باستخدام حلقة for للمرور على عناصر المصفوفة حيث سنبدأ من منتصفها و نعود للبداية؛ و السبب أن النصف الثاني (اليميني) من المصفوفة هو عبارة عن أوراق و نحن لا نحتاج لـ Max_Heapify على الأوراق، لذلك؛ سنستدعي تابع Max_Heapify من أجل كل عقدة أب في الشجرة وصولاً إلى الجذر لنحصل على شجرة Max_Heapify.

استخدامات الـ Heap

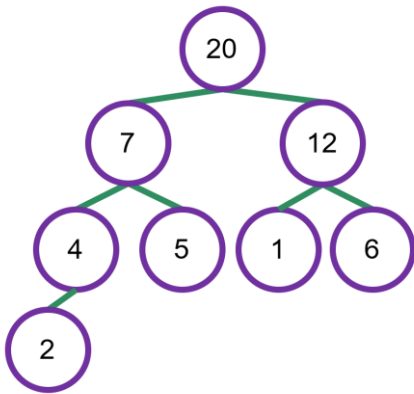
1. Heap Sort:

هي خوارزمية لترتيب العناصر بتبديل الجذر مع آخر عنصر في الشجرة وحذفه تدريجياً. يمكننا أن نبدأ من مصفوفة عشوائية لكن أكبر عنصر فيها سيكون في أول index، نحن نهدف إلى تحقيق مفهوم Max-Heap في الشجرة، ثم نبذل بين الجذر root (أول عنصر في المصفوفة) وآخر عنصر فيها، وضعنا العنصر الأخير في مكانه المناسب، ثم نتجاهل العنصر الأخير من الشجرة (آخر ورقة) فينقص حجم الـ Heap بمقدار واحد. ونتيجةً لعمليات التبديل سيحدث خلل في شرط Max-Heap ولإصلاحه نستخدم خوارزمية Max Heapify وهكذا نستمر بنفس الطريقة.

مثال: لدينا المصفوفة التالية و نريد ترتيبها:

1	4	12	7	5	20	6	2
---	---	----	---	---	----	---	---

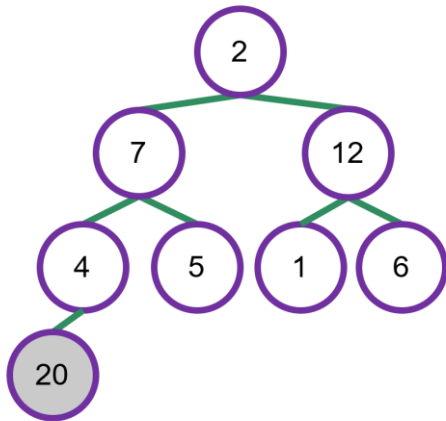
بداية... نبادل بين أكبر عنصر والعنصر الأول في المصفوفة لتحقيق شرط ال Max Heap وبعد عدة عمليات تبديل حصلنا على Max-Heap.



20	7	12	4	5	1	6	2
----	---	----	---	---	---	---	---

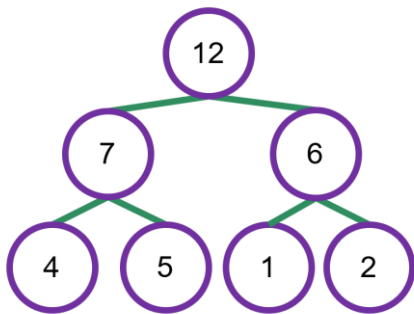
خطوات الترتيب:

1. نبادل بين 20 و 2.



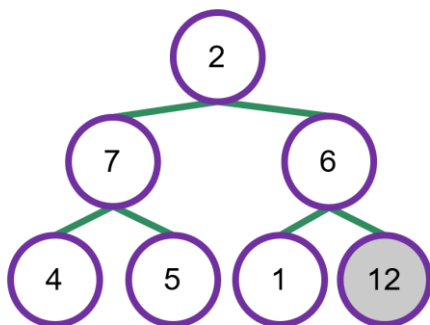
2	7	12	4	5	1	6	20
---	---	----	---	---	---	---	----

2. نتجاهل العنصر الأخير 20 ونعيد ترتيب الشجرة لنحصل على Max Heap.



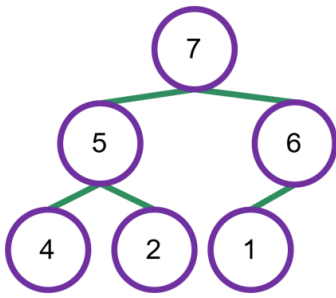
12	7	6	4	5	1	2	20
----	---	---	---	---	---	---	----

3. نبادل بين 2 و 12.



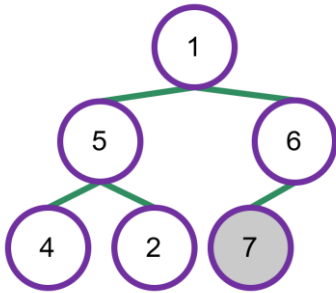
2	7	6	4	5	1	12	20
---	---	---	---	---	---	----	----

4. نتجاهل العنصر الأخير 12 ونعيد ترتيب الشجرة مجدداً باستخدام Max Heapify.



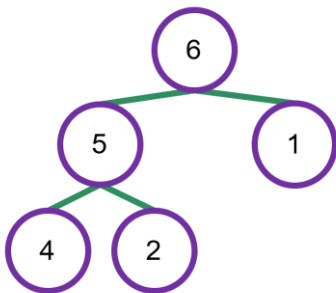
7	5	6	4	2	1	12	20
---	---	---	---	---	---	----	----

5. نبادل بين 7 و 1.



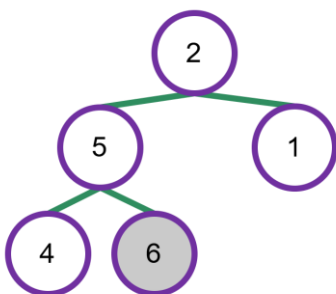
1	5	6	4	2	7	12	20
---	---	---	---	---	---	----	----

6. نتجاهل العنصر الأخير 7 ونعيد ترتيب الشجرة باستخدام Max Heapify.



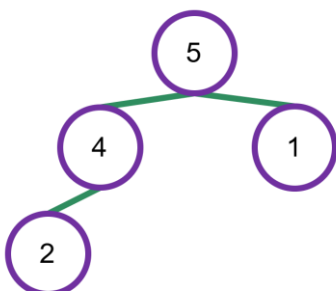
6	5	1	4	2	7	12	20
---	---	---	---	---	---	----	----

7. نبادل بين 6 و 2.



2	5	1	4	6	7	12	20
---	---	---	---	---	---	----	----

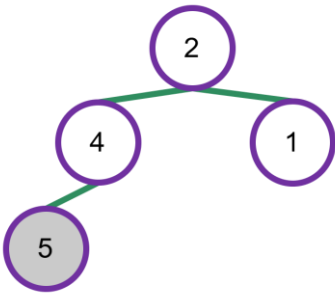
8. نتجاهل العنصر الأخير 6 ونعيد ترتيب الشجرة باستخدام Max Heapify.



5	4	1	2	6	7	12	20
---	---	---	---	---	---	----	----

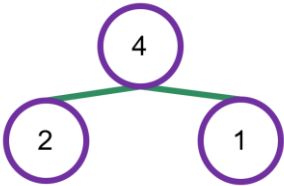
9. نبادل بين 5 و 2.

2	4	1	5	6	7	12	20
---	---	---	---	---	---	----	----



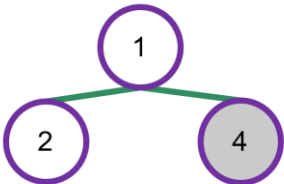
10. نتجاهل العنصر الأخير 5 ونعيد ترتيب الشجرة باستخدام Max Heapify.

4	2	1	5	6	7	12	20
---	---	---	---	---	---	----	----



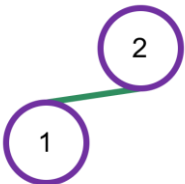
11. نبادل بين 4 و 1.

1	2	4	5	6	7	12	20
---	---	---	---	---	---	----	----



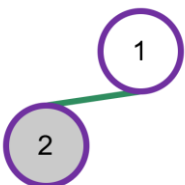
12. نتجاهل العنصر الأخير 4 ونعيد ترتيب الشجرة باستخدام Max Heapify.

2	1	4	5	6	7	12	20
---	---	---	---	---	---	----	----



13. نبادل بين 2 و 1.

1	2	4	5	6	7	12	20
---	---	---	---	---	---	----	----



14. نتجاهل العنصر الأخير 2، و يتبقى لدينا عقدة واحدة فقط.

1	2	4	5	6	7	12	20
---	---	---	---	---	---	----	----

ومنه قد حصلنا على مصفوفة مرتبة بالكامل.

في الحقيقة نحن نرتب المصفوفة من نهايتها إلى بدايتها.

تعقيد Heap Sort:

$$n \log_2 n$$

فالترتيب باستخدام Heap Sort أفضل من عملية الـ Sorting العادية $O(n^2)$.

2. Priority Queue:

تذكرة: ما هو ال Queue؟

هو بنية معطيات خطية تعتمد على مفهوم "First In First Out" أي أن أول عنصر سيدخل إلى المصفوفة هو أول عنصر سيخرج منها.

أما Priority Queue (رتل الأولويات):

هو نوع خاص من الأرتال يكون فيه لكل عنصر أولوية عن غيره من العناصر، فيكون ترتيب العناصر في المصفوفة بحسب الأولوية وليس بحسب ترتيب الدخول.

مفهوم Priority Queue باستخدام Max-Heap:

لدينا ثلاثة توابع أساسية في ال Priority Queue:

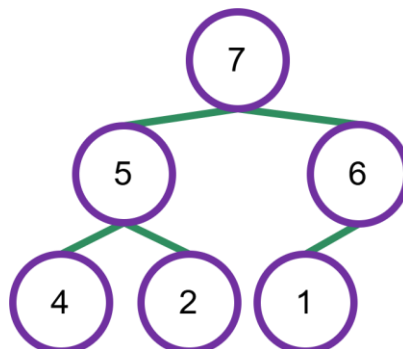
- **تابع `top()`**: يعيد العنصر ذي الأولوية الأعلى دون حذفه، بما أننا نطبق Max-Heap فإن العنصر ذي الأولوية الأعلى هو الأكبر؛ أي أول عنصر في المصفوفة (الجزر).
نستطيع إيجاد العنصر الأول باستخدام تابع `top()` بتكلفة: $O(1)$.

```
1 Top (int[] A) {
2     return A[0];
3 }
```

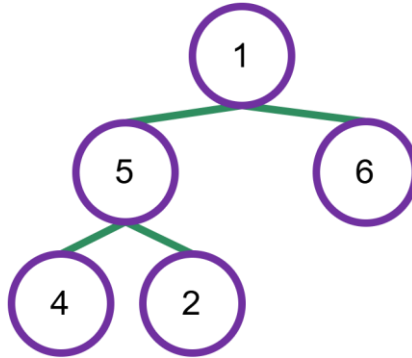
- **تابع `dequeue()`**: يحذف العنصر الأول من المصفوفة، وبما أن العنصر الأول في ال priority queue هو الجزر سنقوم بالمبادلة بين الجزر الأصلي و آخر عنصر في مصفوفة ال Max-Heap و لكن بعد حذف الجزر قد يختل شرط Max-Heap ؛ لذلك نقلل حجم المصفوفة بمقدار واحد و نطبق Max Heapify على بقية المصفوفة و سنحصل بعد كل عملية حذف على ترتيب للمصفوفة باستخدام Max-Heap.

تعقيد هذا التابع هو: $O(\log_2 n)$

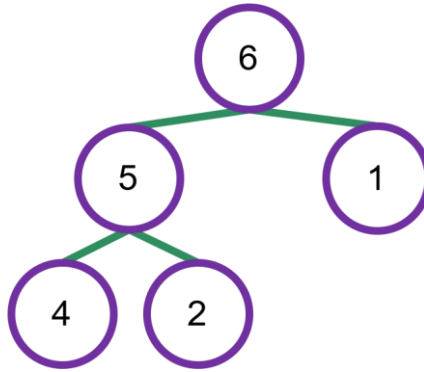
مثال: إذا كانت لدينا Max-Heap التالية:



و قمنا باستدعاء التابع dequeue(): سيتم استبدال الجذر بأخر ورقة على اليمين و من ثمّ حذف هذه الورقة لتصبح الشجرة:



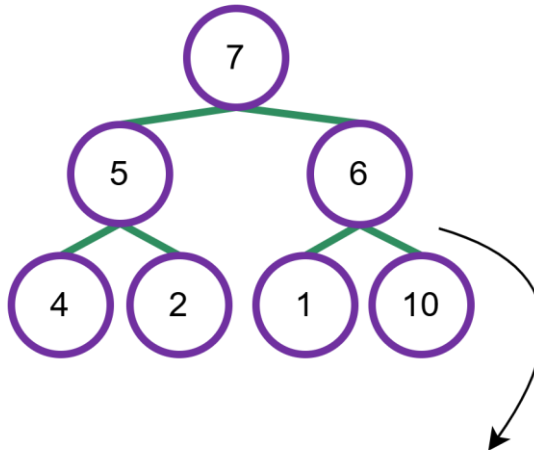
و نعيد ترتيب الشجرة باستخدام Max Heapify:



• **تابع enqueue():** يقوم بإضافة عنصر بحسب أولويّته في المصفوفة، و لأنّ الإضافة تكون على نهاية المصفوفة فلن يتحقق شرط Max-Heap في كلّ Subtree لذلك؛ سنقوم بعملية (swap) على الآباء حتى الوصول إلى المكان المناسب للعنصر (قد يكون المكان المناسب هو الجذر > هو العنصر ذو الأولويّة الكبرى> أو قبل الجذر).

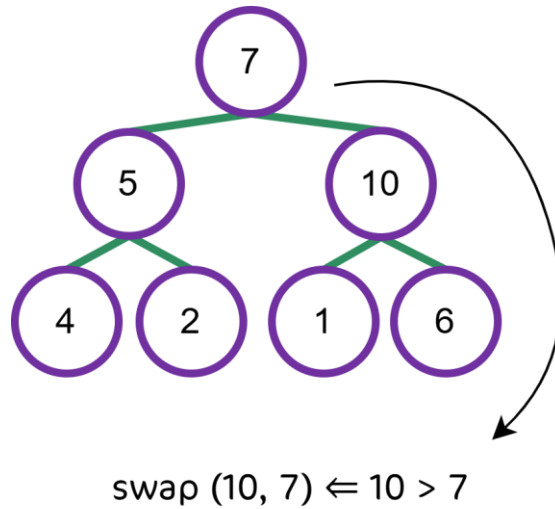
تعقيد هذا التابع هو: $O(\log_2 n)$

مثال: إذا أردنا إضافة (10) على الشجرة الآتية:

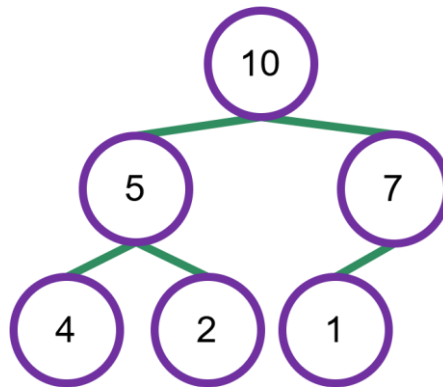


swap (10, 6) $\Leftarrow 10 > 6$

سيصبح شكل الشجرة:



وستكون النتيجة:



وهي Max-Heap.

ملاحظة هامة جداً: الإضافة تكون دائماً في الأوراق.

و نلاحظ أنّ استخدام Priority Queue أقلّ كلفة من استخدام Sorted Array و ذلك لأنّ تعقيد عملية الإضافة في ال Sorted Array هو $O(n)$ بينما في ال Priority Queue هو $O(\log_2 n)$ فقط.

The End...